Winning Is Not Killing: Active Win Rate and Expert Iteration with a Network-Free Kill-Search in Bomberman

Anonymous Author(s)
Affiliation withheld for review

Abstract—Agents for Bomberman-like games are usually ranked by win rate, yet a game can be won because the opponent destroys itself or is caught by a shrinking arena. We define active win rate (ActiveWR), the share of games won with at least one kill credited to the learner's own bomb, and use it to study how a hand-written kill-search that uses no neural network can be turned into a fast neural policy on consumer hardware. In a 9×9 one-versus-one Bomberman game against a fixed rule-based bot, policy-gradient training with a safety mask and auxiliary teachers reaches 40.0–51.0% ActiveWR, and more than a dozen standard refinements fail a gate fixed before testing. A kill-search that looks one, two or three learner moves ahead over 12 ticks, with the opponent modelled as the rule-based bot, lifts the same network to 64.75, 82.0 and 94.0%. Used as a sparse auxiliary loss inside reinforcement learning the search does not help (-2.75 pp), whereas dense distillation of its labels on decision states does ($+16.25$ pp). Three rounds of expert iteration, each distilling the current student plus a deeper search, produce S3: a plain convolutional policy at 92.25% ActiveWR, 0.75 pp below its search teacher with a confidence interval of [-4.0, +2.5] that includes zero, using one forward pass per move. Interventions that add 3.75–16 pp of win rate add no ActiveWR. Against frozen neural opponents most games end in draws; with a correct opponent model the search finds a kill plan in only 15.9% of them, so most of these draws seem to come from the game rather than from missed chances. All tests use single-use seeds, a paired bootstrap over boards and acceptance gates fixed in advance.

Index Terms—Bomberman, Pommerman, reinforcement learning, expert iteration, policy distillation, game-tree search, evaluation metrics

I. Introduction

Deep reinforcement learning (RL) reaches human or grandmaster level in Atari and StarCraft II [1], [2], but with compute far beyond a single consumer machine. Bomberman, and its research variant Pommerman [3], is a hard test for model-free RL at small scale. A bomb placed at random kills its owner about 40% of the time [4], so exploration is suicidal; the reward for a kill arrives many steps after the decision that made it; and the decisive moments are rare. Large RL efforts needed hundreds of CPUs [5] or long population-based training [6], and still lost to search-based agents [6], [7]. The strongest agents combine an action filter or a search with learning [7]–[9].

A second problem is less visible: how agents are scored. Several groups noticed that an agent can collect wins when the opponent kills itself [6], [8]–[10], but results are still reported as win rate (WR) or tournament points. We met this problem directly. An early checkpoint of ours won 52.5% of 40 four-player games while killing nobody with its own bombs, and a mask margin that raised WR by 10.75 pp left the kill-based win rate unchanged (Section VI-A).

This paper reports a controlled study, run on consumer hardware, of how to get a fast neural Bomberman agent that wins by killing. We make the following contributions.

- C1, a metric. We define ActiveWR, the probability of winning and killing at least one opponent with one's own bomb, with kill credit assigned to the bomb's owner through chain explosions. Win rate overstates three of our interventions by 3.75–16 pp (Fig. 8).
- C2, three uses of the same search. With one kill-search, we compare (a) sparse search labels as an auxiliary RL loss, (b) search at play time and (c) dense distillation of search labels. Only (b) and (c) help (-2.75, $+16.5$ and $+16.25$ pp). This tests the two future directions proposed by Kartal et al. [10].
- C3, expert iteration with a network-free expert. Three rounds of "student plus deeper search $\rightarrow$ distil" raise a plain CNN from 48.25% (best RL policy) to 92.25% ActiveWR; the student–teacher gap shrinks from -3.75 to -3.5 to -0.75 pp while the teachers get stronger.
- C4, controlled negative results. We report gated, single-test-seed results for generalized advantage estimation (GAE), PPO clipping, advantage normalisation, reward changes, curricula, sudden-death training, group-relative advantages and KL-anchored RL after distillation. None passes.
- C5, a draw analysis. Strong agents draw most games against each other. A probe with the correct opponent model finds a kill plan in 15.9% [9.9, 22.7] of such draws, against 90.3% under the rule-bot assumption, and the wrong opponent model costs 33 pp.
- C6, transfer to a larger board. Re-distilling on 11×11 adds $+41.0$ pp [34.5, 47.25] against the rule bot but only $+3.83$ pp [1.92, 5.83] against neural opponents.

Fig. 1 summarises the path from the first RL checkpoint to the final agent. Each building block (masked policy gradient, search, distillation, group-relative advantages) is known; the contribution is the metric, the controlled

comparison under a fixed protocol, and the analysis of where the remaining draws come from.

II. Related Work

RL in Bomberman and Pommerman. Pommerman [3] made Bomberman a multi-agent benchmark. Without action filtering, PPO learned nothing in five days; with a filter it learned [4]. Skynet [8] combined PPO, an action filter and reward shaping, and observed that wins often came from opponents killing themselves. Imitation then RL suffered catastrophic forgetting [11]; curriculum and population-based self-play still lost 70–95% of games to search agents after 200 million steps [6]. Backplay [12] starts episodes from the end of a demonstration and moves the start backwards, and continual match training used 220 CPUs and 32 GPUs [5]. For classic Bomberman, Q-learning with a multi-layer perceptron was used to compare exploration rules [13], and imitation followed by PPO was applied to a Unity version [14].

Search in Pommerman. The NeurIPS 2018 winner used depth-limited tree search with pessimistic opponent scenarios and beat the best RL agents in 87.1% of 22,500 games [7]. Statistical forward planning (MCTS, RHEA) produced 59–84% draws between strong planners [15]. MCTS with learned opponent models can turn defensive and slow [9]; mixtures of phase experts help MCTS by 5–11% in Pommerman [16]. Opponent modelling inside the policy was studied by DOMAC [17].

Search as a teacher. Guo et al. [18] trained real-time Atari CNNs on UCT decisions. Expert iteration (ExIt) [19] and AlphaGo Zero / AlphaZero [20], [21] alternate a search expert and a network apprentice, with the network guiding the search. In Pommerman, MCTS was used as a demonstrator inside A3C [22] on an 8×8 mini-board [10], [23], [24]; those authors proposed, but did not test, calling the demonstrator only in relevant states and storing its actions in a buffer. Distillation [25]–[27], DAgger [28] and kickstarting [29] give the supervised side of our method.

Safety and evaluation. Masking invalid actions in the logits keeps a valid policy gradient [30]; shields block unsafe actions with minimal interference [31]. Only potential-based shaping preserves optimal policies [32], and dense rewards should be annealed [33]. RL results need confidence intervals over independent units and several runs [34], [35]; we use Efron's bootstrap [36] over paired boards. Table I positions this work.

TABLE I
Positioning against the closest work.

Work	Learning	Search	Metric
[4], [8]	PPO + action filter	–	WR
[7]	–	depth 1, pessimistic	WR, draws
[10], [24] (8×8)	A3C + MCTS demonstrator	shallow MCTS	reward curve
[9]	ExIt (AlphaZero-style)	MCTS + network	WR
[6]	PPO, curriculum, PBT	–	WR, enemies killed
Ours (9×9, 11×11)	PG, dense distillation	network-free kill-search	ActiveWR, paired CI

III. Problem Setting

Game. We use a NumPy re-implementation of Pommerman-style free-for-all Bomberman. The board is 9×9 with a border and pillars of rigid walls on (even, even) cells; 70% of the other free cells hold wooden walls, except 3×3 spawn zones. Two agents start in opposite corners. Actions are STOP, four moves and BOMB. A bomb explodes after 8 steps into a cross-shaped blast (initial range 2), destroys wooden walls (a power-up appears with probability 0.3: extra bomb or longer range), kills agents in the blast and chains into other bombs. Flames last 2 steps. A game ends when at most one agent is alive or after 500 steps (draw). Fig. 2 shows a game in progress. Our default sudden-death schedule E closes the arena from the border inwards from step 475 every 25 steps, with a 40-step warning.

Observation. Each agent sees 13 9×9 planes (Fig. 13, Appendix): rigid and wooden walls, bomb timers, flames, two power-up types, own and opponent positions, three constant planes (free bomb slots, capacity, range), the inverse time until each cell is hit by a forecast blast including chains, and the sudden-death warning.

Safety mask and opponent. Every agent acts through the same safe-action mask $m(s) \in \{0,1\}^6$, which removes moves into cells that a forecast blast will reach before the agent can leave, and bombs without an escape route; it has fallbacks so that at least one action stays legal. The standard opponent is a rule-based bot (RuleBot) that dodges, collects items, attacks with the correct blast range and uses the same mask; when none of its rules applies it picks a random safe move, so its play is stochastic. RuleBot is kept fixed throughout.

Metric. Let W be the event that the learner is the last agent alive and K the event that at least one opponent died in a blast credited to a learner bomb (credit follows the owner of the bomb that started a chain). We report

$$\mathrm{ActiveWR}(\pi) = \Pr_{\pi}[\, W \wedge K \,], \tag{1}$$

and decompose every evaluation into active wins, passive wins (opponent self-kill), ring wins (opponent caught by sudden death), draws, learner self-kills and other losses. $\mathrm{WR} = \Pr[W]$ is reported only as a secondary number.

IV. Method

A. Network

All agents share one actor–critic CNN (Fig. 3): three 3×3 convolutions (32, 64, 64 channels, padding 1, ReLU), a 256-unit fully connected layer, a 6-way actor head and

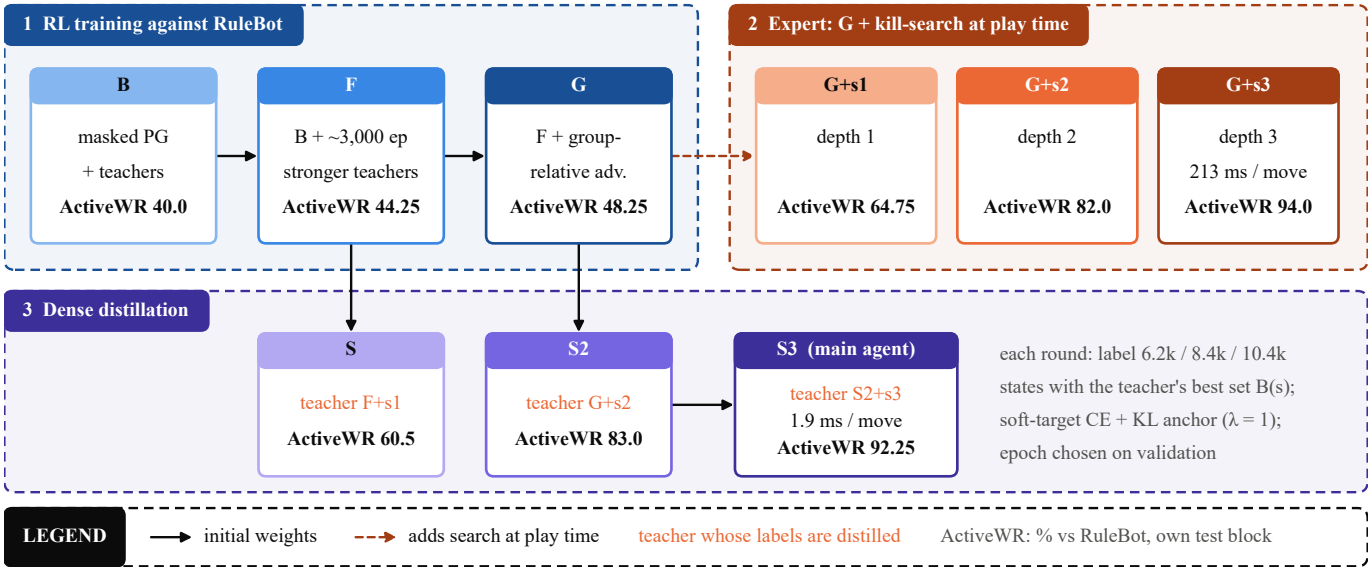


Fig. 1. Path from the first RL checkpoint to the final agent. Panel 1: RL-trained CNNs (blue). Panel 2: G with kill-search at play time (orange, the expert); the dashed arrow adds search without changing any weights. Panel 3: distilled CNN students (violet). Black arrows give the initial weights of each model; the orange label names the teacher whose search labels it is trained on, so S2 starts from G, not from S. Each agent keeps the same colour in every figure. Numbers are ActiveWR (%) against RuleBot under schedule E, each on the model's own single-use test block (Table III); only same-block comparisons are inferential.

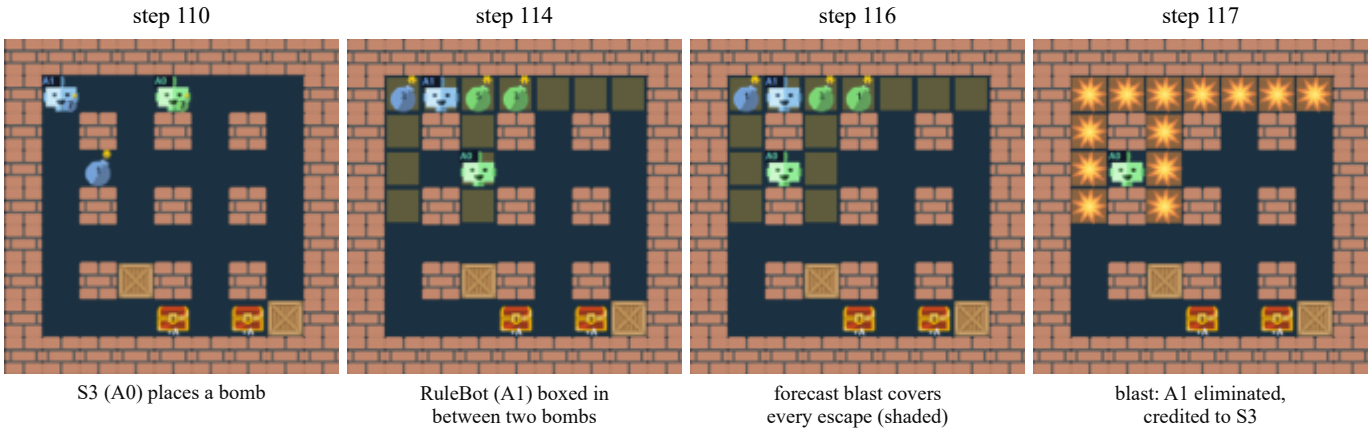


Fig. 2. Four frames of a real game, captured from the game's own viewer: S3 (A0, green) against RuleBot (A1, blue) on validation board 21000. The number on a bomb is its fuse; shaded cells are the forecast blast area (plane 11 in Fig. 13), shown in the middle two frames. S3's bomb placed at step 110 and a second one at step 112 box RuleBot in next to its own bomb; the chain explosion at step 117 kills it, and the kill is credited to S3, so this game counts as an active win.

a scalar critic head, 1,388,359 parameters in total. The policy masks logits before the softmax,

$$\pi_\theta(a \mid s) = \frac{m_a(s)\exp z_a(s)}{\sum_b m_b(s)\exp z_b(s)}, \tag{2}$$

implemented as masked_fill$(\neg m, -10^9)$ both when sampling and inside every loss, which keeps the policy gradient valid [30]. Actions are sampled; on validation, argmax play lowered F's ActiveWR by 9.25 pp [-16.0, -2.5] and raised draws by 7.5 pp [1.25, 13.75]. We did not diagnose why; a likely reason is that a deterministic policy can repeat the same move sequence, which sampling breaks.

B. Stage 1: masked policy gradient with auxiliary teachers

The RL stage trains against RuleBot with Monte-Carlo returns $G_t = \sum_k \gamma^k r_{t+k}$, $\gamma = 0.99$, and a learned baseline (REINFORCE with baseline [37], [38]; GAE [39] and PPO clipping [40] were tested as variants):

$$\mathcal{L}_{\mathrm{RL}} = \tfrac{1}{T}\sum_t \Big[-\log \pi_\theta(a_t \mid s_t)\,\hat{A}_t + c_v \hat{A}_t^2 - c_e\,\mathcal{H}\big(\pi_\theta(\cdot \mid s_t)\big)\Big] + \sum_k c_k\,\mathcal{L}_k^{\mathrm{teacher}}, \tag{3}$$

with $\hat{A}_t = G_t - V_\phi(s_t)$ (detached in the policy term). The auxiliary terms are cross-entropy losses towards four rule-based teachers (escape, combat, navigation, ring safety)

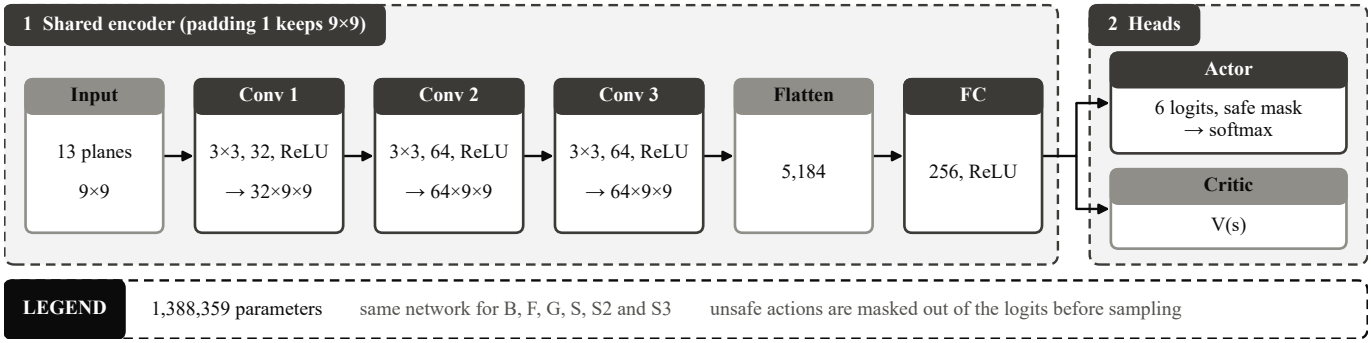


Fig. 3. Actor–critic network used by every agent in this paper (B, F, G, S, S2, S3). The safe-action mask is applied to the logits.

with weights $0.5, 0.9, 0.4, 0.6$; the attack-related weights are multiplied by 1.75 after step 350 of a game. Rewards are event-based: -0.001 per step alive, $+0.05$ per useful item, $+0.15$ per wooden wall destroyed, $+4$ per credited kill, -1 on death and an extra -1.5 for a self-kill. The environment's $+10$ for being last alive is kept only for active wins; for passive wins it is cancelled, which aligns the return with ActiveWR rather than shaping it.

Group-relative advantage. Because a game of about 230 steps contains on average only about 2.2 steps where the actions' search scores differ, and about 1.25 where some action leads to a kill, we also tried a GRPO-like update [41] at decisive states, where the kill-search below finds a credited kill within 12 ticks. From each such state the current policy is replayed $K = 16$ times for $H = 20$ ticks against fresh RuleBots, with outcome $o_k \in \{+1 \text{ (credited kill)}, -1 \text{ (death)}, 0\}$ and advantage $A_k = o_k - \bar{o}$; the loss $-\frac{1}{K}\sum_k A_k \sum_t \log \pi_\theta(a_{k,t} \mid s_{k,t})$ is added with weight 4 (at most 6 states per episode, at least 4 steps apart). The resulting checkpoint G is our best RL policy.

C. Stage 2: network-free kill-search

The expert is a short-horizon search that uses no network (Algorithm 1). For each legal first action, a copy of the game is simulated for 12 ticks (bomb fuse 8 + flame 2 + slack): the opponent plays RuleBot and the learner only dodges after its first action. A rollout scores $+1$ if an opponent dies from a learner bomb while the learner is alive, -1 if the learner dies, else 0. Depth $d = 2$ or 3 extends the search to two or three learner moves before dodging, and is only expanded when no shorter plan kills. The search is skipped unless an opponent is within Manhattan distance 6 (no kill fits the horizon beyond it). The CNN's sampled action is kept unless another action scores strictly higher, so the search only changes decisions that matter.

Algorithm 1 Kill-search of depth d (play time)

Require: state s, learner i, mask m, CNN action a_0, depth d, horizon $H = 12$
if no opponent within distance 6 then return a_0
end if
for a with $m_a(s) = 1$ do
  $v_1(a) \leftarrow \text{Rollout}(s, [a], H)$ ▷ RuleBot opponent, learner dodges
end for
if $\max_a v_1(a) > v_1(a_0)$ then return $\arg\max_a v_1(a)$
end if
for $\ell = 2, \ldots, d$ do ▷ only if no shorter kill
  for safe action prefixes $(a, b_2, \ldots, b_\ell)$ do
    $v_\ell(a) \leftarrow \max \text{Rollout}(s, (a, b_2, \ldots, b_\ell), H)$
  end for
  if some $v_\ell(a) > 0$ and $v_\ell(a_0) \le 0$ then return first move of a kill plan
  end if
end for
return a_0

D. Stage 3: dense distillation and expert iteration

Data. The teacher (current student + kill-search of depth d) plays 3,000 games against RuleBot on fresh boards (sudden death E in 75% of games). At every learner step the search scores all legal actions; a decision state is one where the scores differ. Its target spreads mass uniformly over the best set $B(s)$:

$$q(a \mid s) = \frac{\mathbb{1}[a \in B(s)]}{|B(s)|}, \qquad B(s) = \arg\max_{a:\, m_a(s)=1} v(a). \tag{4}$$

With probability 0.05 an ordinary state is also stored as an anchor, with the reference logits of the starting network π_{ref}. Rounds 1–3 collected 6,171, 8,449 and 10,425 decision states and up to 29,483 anchors.

Loss. The student starts from the previous network and minimises

$$\mathcal{L}_{\text{distill}} = \mathbb{E}_{s \in \mathcal{D}_{\text{dec}}}\Big[-\textstyle\sum_a q(a \mid s) \log \pi_\theta(a \mid s)\Big] + \lambda\, \mathbb{E}_{s \in \mathcal{D}_{\text{anc}}} \mathrm{KL}\big(\pi_{\text{ref}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s)\big), \tag{5}$$

with $\lambda = 1$, Adam, learning rate 10^{-4}, batch 256, 8 or 16 epochs. A board-level 10% split is held out to monitor accuracy, and the epoch is chosen by ActiveWR on validation seed 21000 before any test. We do not augment with rotations: on 6,171 audited states only 79.2% kept the same best-action set after a symmetry

TABLE II
Main settings (defaults; one train seed per arm).

RL (Stage 1)	Adam, lr 2.5×10^{-5}, $\gamma = 0.99$, $c_v = 0.5$, $c_e = 0.0075 \to 0.0015$; 3,000 episodes per arm, validation every 500 (400 games)
Group advantage	$K = 16$, $H = 20$, weight 4, $\leq$6 states/episode
Kill-search	horizon 12 ticks, reach 6, depth 1/2/3, opponent model RuleBot
Distillation	3,000 teacher games, anchors 5%, $\lambda = 1$, lr 10^{-4}, batch 256
Evaluation	200 boards $\times$ 2 corners, 500 steps, schedule E, sampled actions
Statistics	paired bootstrap over boards, 10^4 resamples, seed 20261014
Hardware	consumer GPUs (RTX 3050/4060) for updates; CPU for search and tests

transform, because the search and RuleBot break ties asymmetrically (cf. AlphaZero dropping augmentation for asymmetric games [21]).

Iteration. Round 1 distils F + depth-1 search into S (from F); round 2 distils G + depth-2 into S2 (from G); round 3 distils S2 + depth-3 into S3 (from S2). Unlike ExIt [19], the expert never uses the network's prior or value; the student improves only because the expert also contains the previous student's own moves where the search is indifferent, which keeps the data close to the student's state distribution [28].

V. Evaluation Protocol

Every experiment follows the same pre-specified protocol (fixed in our repository before each run, not in a public registry). (i) Configuration, source hashes and the selection rule are written before training. (ii) Checkpoints are selected only on validation seed 21000. (iii) The selected models are tested once on a fresh block of 200 boards, each played from both corners (400 games per model); a test seed block is never reused. (iv) Models are compared with a paired bootstrap [36]: for board b, d_b is the difference in ActiveWR averaged over the two corners, $\hat{\Delta} = \bar{d}$, and the 95% percentile interval comes from 10^4 resamples of boards. (v) A change is accepted only if $\hat{\Delta} \geq +5$ pp, the lower CI bound is above zero, and self-kills do not rise by more than 1 pp. Failing results are reported as such. A single 400-game test block of the same pair varies by about ± 5 pp across blocks (e.g. -6.5, $+3.5$, $+3.0$ for one pair on three blocks; 0.0 [-3.75, 3.75] pooled over 600 boards), which is why the gate requires both a margin and a positive lower bound. We ask five research questions: RQ1 how much does WR misjudge agents? RQ2 which use of a search transfers its strength to a CNN? RQ3 how close can expert iteration with a network-free expert bring a CNN to the search? RQ4 why do RL refinements fail from a good policy? RQ5 are draws between strong agents missed chances or structural?

VI. Results

A. RQ1: win rate misjudges agents

Fig. 8 shows four interventions measured with both metrics on paired boards. A learner-side mask margin of $k = 15$ steps for the shrinking ring raised WR by $+10.75$ pp [6.25, 15.5] but ActiveWR by only $+0.75$ [-1.25, 2.75]; deaths in the ring fell from 116 to 76 of 400, so the gain was survival, not attack. A rule that makes the learner stand on the opponent's escape cell raised WR by $+3.75$ [1.75, 6.0] while ActiveWR fell by 1.75 [-4.25, 0.75], with passive wins up from 9.5 to 15.25%; owner-based credit shows that the rule turned 16 of F's active wins into passive ones. Turning sudden death on adds 16 pp of WR to G with no ActiveWR change, and does nothing for S3, which wins before the ring starts. The earliest example is starker: a four-player checkpoint that passed our original WR gate (52.5% over 40 games) had zero kills with its own bombs. Evaluation only against RuleBot also overstates search agents: G+s3 reaches 94% ActiveWR against RuleBot (86–94% in three- and four-player games) but only 8.5–20.5% against frozen CNNs, with 50–70% draws.

B. RQ2: three uses of the same search

Table IV compares three ways to use the depth-1 kill-search. As a sparse auxiliary loss inside RL (labels only where the search disagrees with the CNN, $\approx$0.9 labels per episode, each used once) it did not help: -2.75 pp [-9.75, 4.0] over 3,000 episodes, no better than plain RL (-6.5 [-12.75, 0.0]). At play time it adds $+16.5$ pp [12.5, 20.5]. Densely distilled (about 6,200 decision states with counter-examples, 8 epochs), it moves the CNN by $+16.25$ pp [9.75, 22.5], nearly the play-time gain, with no search at play time. The distilled student learned the condition "only BOMB kills here": the probability of the best move on held-out decision states rose from 0.52 to 0.91, from 0.28 to 0.93 on states where the learner must stand in the escape cell, while P(BOMB) on ordinary states stayed at $0.088 \to 0.095$.

C. RQ3: expert iteration with a network-free expert

Search depth. Fig. 10 links depths on shared test blocks: depth 1 adds +16.5 over G, depth 2 adds +21.0 [17.0, 25.25] over depth 1, and depth 3 adds +8.25 [5.25, 11.5] over depth 2, nearly removing draws ($5.75 \to 0.25$%). The gains come from creating a kill (step, then bomb) rather than finishing one: depth-1 search plateaus near 65% whatever CNN it wraps (F, G or S).

Iterated distillation. Each round closes most of the student–teacher gap (Fig. 4, bottom group): S $-$ (F+s1) $= -3.75$ [-10.0, 2.25], S2 $-$ (G+s2) $= -3.5$ [-8.25, 1.25], S3 $-$ (S2+s3) $= -0.75$ [-4.0, 2.5] (versus G+s3: -1.75), while the teachers get stronger. Round gains are +16.25 (S $-$ F), +35.0 [29.0, 41.25] (S2 $-$ G) and +10.5 [6.5, 14.5] (S3 $-$ S2). The S3 gain is active: passive wins change by -1.5 [-4.5, 1.5], draws by -6.25 [-9.0, -3.75], games are

TABLE III

Main results against RuleBot (9×9, 1v1, schedule E, 400 test games per model). Each row uses the model's own single-use test block; rows on the same block can be compared directly (Fig. 4). CI: 95% bootstrap over boards. *B's first confirmation test (block 32000) was run before sudden death existed and gave 32.75 [28.25, 37.5] ActiveWR with 60.5% draws; the row shown is B under schedule E on the cross-check block 60000.

Agent	Description	Test block	ActiveWR % [95% CI]	WR %	Passive/ring win %	Draw %	Self-kill %	Mean steps
B*	masked PG + teachers	60000	40.0 [35.0, 45.0]	64.0	24.0	22.5	0.5	357
F	B + ∼3,000 RL episodes	43000	44.25 [39.25, 49.25]	63.5	19.25	25.0	1.0	340
G	F + group-relative advantage	46000	48.25 [43.5, 53.25]	69.0	20.75	23.25	0.75	335
G+s1	G + kill-search, depth 1	46000	64.75 [60.25, 69.25]	84.0	19.25	15.25	0.0	287
G+s2	G + kill-search, depth 2	47000	82.0 [78.25, 85.75]	93.75	11.75	5.75	0.0	208
G+s3	G + kill-search, depth 3	49000	94.0 [91.75, 96.25]	99.5	5.5	0.25	0.0	114
S	distil F+s1 (round 1)	43000	60.5 [55.25, 65.75]	78.5	18.0	18.0	1.0	279
S2	distil G+s2 (round 2)	48000	83.0 [79.5, 86.5]	90.25	7.25	7.0	0.25	211
S3	distil S2+s3 (round 3)	57000	92.25 [89.5, 94.75]	98.0	5.75	0.5	0.25	131

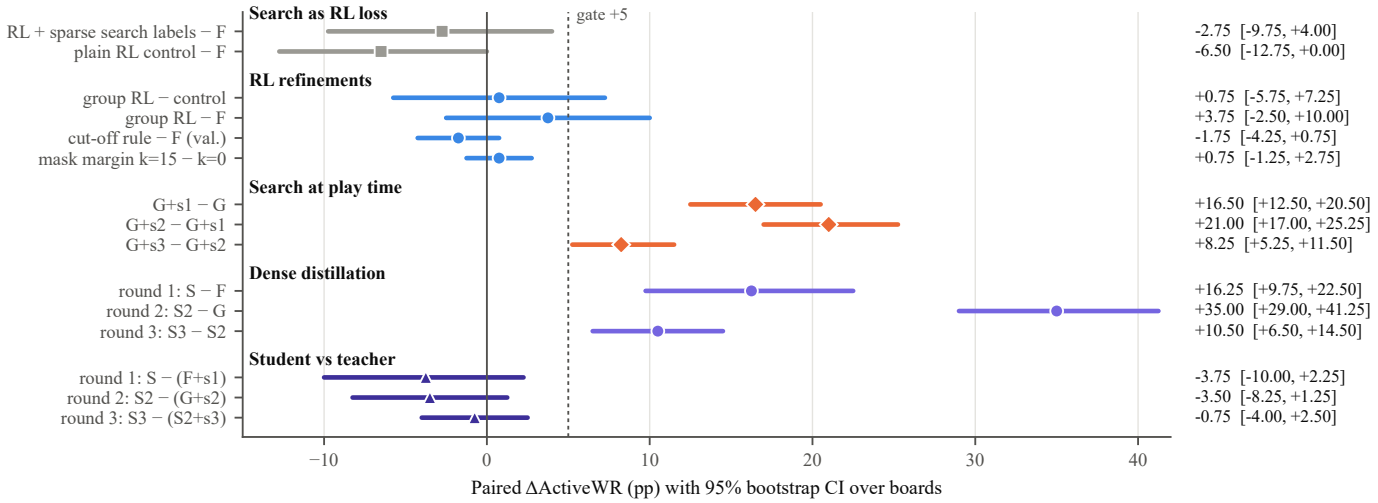


Fig. 4. Paired ΔActiveWR with 95% bootstrap CIs over boards, recomputed from per-game logs with the protocol's bootstrap seed; every value with a logged counterpart matches it exactly. Each group borrows the colour of the agent family it concerns. Dashed line: +5 pp gate. "val." marks a validation-block comparison.

TABLE IV

Three uses of the same depth-1 kill-search (paired ActiveWR differences, own test blocks).

Use	Comparison	ΔActiveWR [95% CI]	Block
Sparse auxiliary loss in RL	search arm − F	−2.75 [-9.75, +4.0]	42000
Search at play time	G+s1 − G	+16.5 [+12.5, +20.5]	46000
Dense distillation	S − F	+16.25 [+9.75, +22.5]	43000

shorter (131 versus 202 steps), and disabling sudden death gives the same +10.75 [7.0, 14.5]. Round 3 is stable across training seeds: we repeated it twice with new collection boards and new train seeds, and on a shared fresh test block (400 games per model) the three S3 replicates beat S2 by +10.5 [5.75, 15.25], +9.0 [4.25, 13.5] and +9.25 [4.75, 13.75] (mean +9.6, s.d. 0.8); all three pass the gate. Fig. 5 shows the training signals: held-out agreement with the teacher's best set rises from 60.5 to 94.0% in round 2 and from 74.8 to 89.8% in round 3, while the KL to the reference policy on anchor states stays below 0.11 nats. Fig. 9 gives the full action confusion on held-out decision states with a unique best action; in round 2 the main fix is movement: G often walks the opposite way to the search (LEFT in 67% of states where RIGHT is best, UP in 41% where DOWN is best) and almost never stops when STOP is best, while BOMB is already right in 94% of cases. A stronger base does not make the search stronger: S2+s3 versus G+s3 on validation is −5.0 [-11, 1], consistent with a ceiling set by a search that ignores the network.

Cost. A search game takes 23–27 s of CPU time against about 1 s for CNN-only games; per decision, the depth-3 search costs 213 ms on average against 1.9 ms for one S3 forward pass and sample (about 115×; 745 decisions in 6 validation games, one CPU thread, median 132 versus 1.8 ms).

D. RQ4: ablations and controlled negative results

Table V collects every gated comparison; Fig. 4 plots the paired intervals recomputed from per-game logs. From B, 200-episode arms with teacher variants, curricula, advantage normalisation, GAE, reward changes and PPO clipping (evaluated before sudden death was added) all

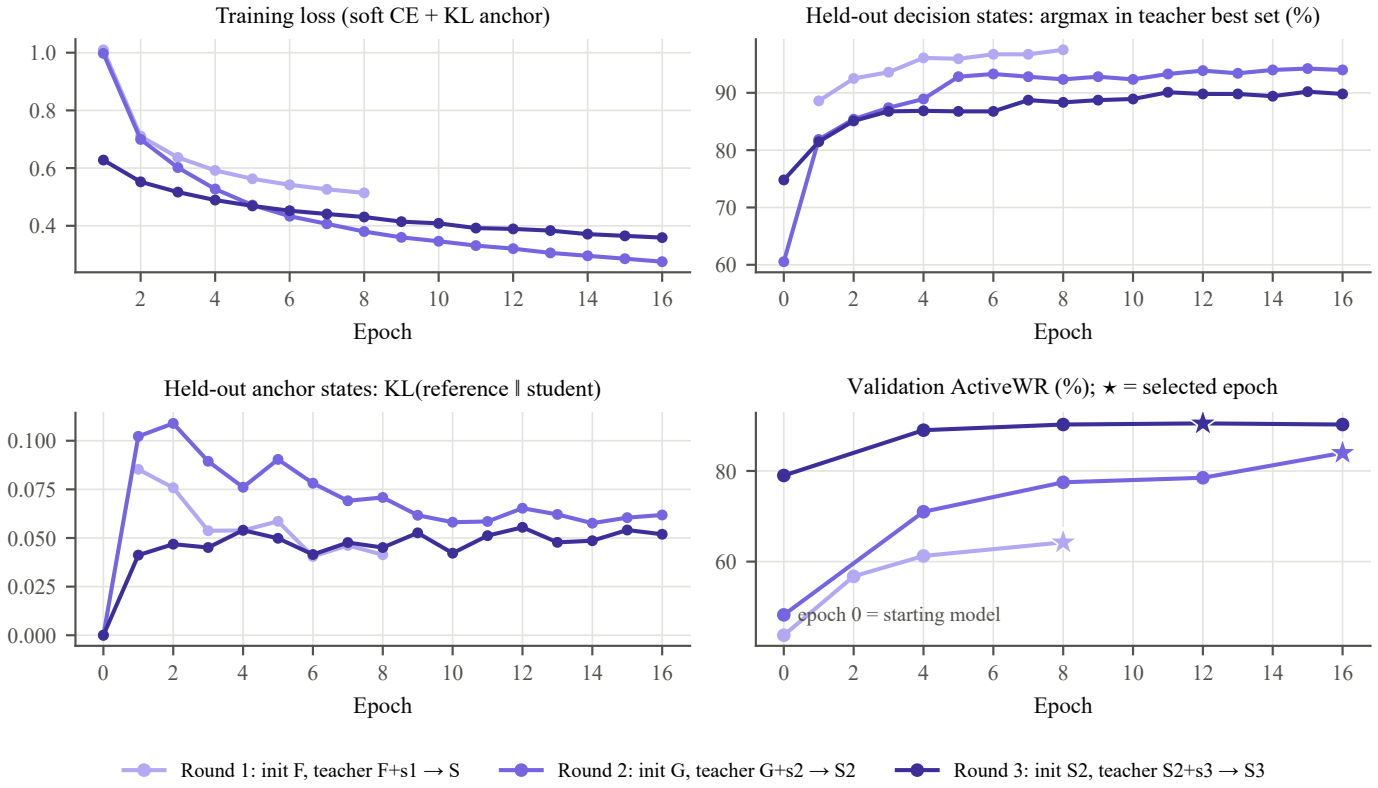


Fig. 5. Distillation rounds, coloured by the student produced. Top left: training loss (soft-target cross-entropy plus the KL anchor). Top right: share of held-out decision states where the student's argmax is in the teacher's best set. Bottom left: KL from the reference (starting) policy on held-out anchor states. Bottom right: validation ActiveWR; epoch 0 is the starting model and stars mark the epochs selected on validation before testing (8, 16 and 12). Held-out sets differ between rounds, so levels are comparable only within a round.

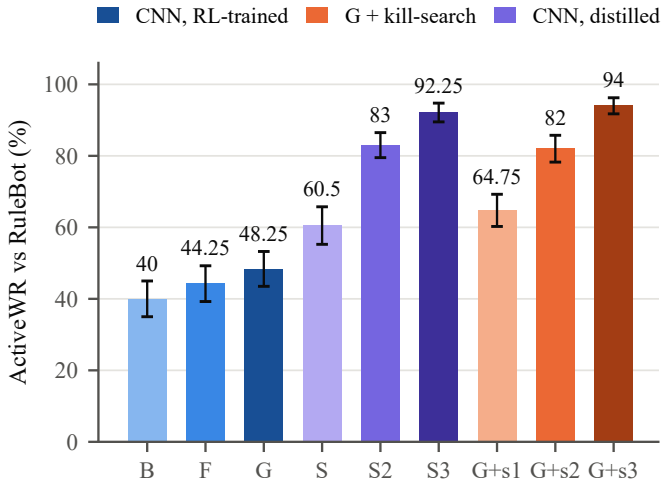


Fig. 6. ActiveWR of every milestone (schedule E, 400 games each) with 95% bootstrap CIs over boards. Each bar uses the model's own test block (Table III), so differences between bars are descriptive. Colours identify agents throughout the paper.

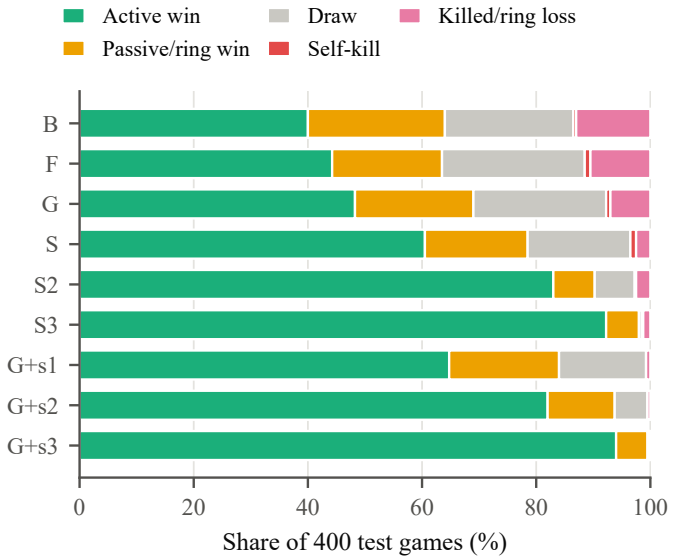


Fig. 7. Outcome composition of the same 400-game tests. RL checkpoints win many games passively or by the ring; distilled students and search agents win by killing. Self-kills stay at or below 1%.

stayed within a few points of B; for GAE and the two-bomb teacher, validation kept the starting weights. Training under sudden death, with or without a ring channel, lowered ActiveWR by 2.25 pp. From F, 3,000 more RL episodes did not help (−6.5), and neither did group-

TABLE V

Ablation study and controlled negative results (ActiveWR in pp; paired over the stated block unless marked). Gate: $\Delta \geq +5$ and CI lower bound > 0. "val." = validation block 21000, descriptive only. Arms from B ran 200 episodes with learning rate 10^{-4} and were evaluated before sudden death existed; the sudden-death rows used an earlier ring schedule (start 400, interval 40). Arms from F ran 3,000 episodes with the defaults of Table II and were evaluated under schedule E.

Group	Change (one variable)	From	Block	ΔActiveWR [95% CI]	Outcome
RL refinements from B (200 ep)	local attack teacher	B	26000	+1.5 [-4.5, 7.25]	fail
	two-bomb teacher	B	26000	0.0 (best = start)	fail
	plain RL control (best ep. 400)	B	32000	+2.25 [-3.25, 8.0]	fail
	combat MOVE guidance	B	33000	+5.25 [-1.5, 12.0]	fail (CI)
	advantage normalisation	B	34000	+0.25 [-5.5, 6.0]	fail
	GAE ($\lambda = 0.95$)	B	35000	0.0 (best = start)	fail
	half wall reward	B	36000	+0.5 vs. B (35.0 vs. 34.5)	fail
	PPO clip $\epsilon = 0.2$, 4 epochs	B	37000	−5.5 (32.0 vs. 37.5)	fail
Sudden death	train under SD, 12 channels	B	40000	−2.25 [-8.0, 3.5]	fail
	train under SD + ring channel	B	40000	−2.25 [-7.5, 3.25]	fail
	learner mask margin $k = 15$	B	41000	+0.75 [-1.25, 2.75] (WR +10.75)	fail
	rule teachers from step 360	B	val.	−4.25 to −2.25	fail
From F (3,000 ep)	plain RL	F	42000	−6.5 [-12.75, 0.0]	fail
	sparse search labels in RL	F	42000	−2.75 [-9.75, 4.0]	fail
	argmax instead of sampling	F	val.	−9.25 [-16.0, -2.5]	worse
	escape-cell cut-off rule	F	val.	−1.75 [-4.25, 0.75] (WR +3.75)	fail
	group advantage $K = 8$, weight 1	F	44000	+0.25 [-5.75, 6.25]	fail
	group advantage $K = 16$, weight 4 (= G)	control	45000	+0.75 [-5.75, 7.25]	fail
After distillation	self-play RL vs. CNN pool, 2,000 ep	S2	val.	83.75 → 65.0 / 57.5	skill lost
	global KL anchor $\beta = 1$, 500 ep	S2	val.	84 → 68 (control 76)	fail
	search opponent model = own CNN	G+s3	val.	−33.0 [-41, -25] vs. RuleBot	rejected
Accepted	kill-search depth 1 at play time	G	46000	+16.5 [12.5, 20.5]	pass
	depth 2 vs. depth 1	G+s1	47000	+21.0 [17.0, 25.25]	pass
	depth 3 vs. depth 2	G+s2	49000	+8.25 [5.25, 11.5]	pass
	distillation rounds 1 / 2 / 3	F / G / S2	43k/48k/57k	+16.25 / +35.0 / +10.5	pass

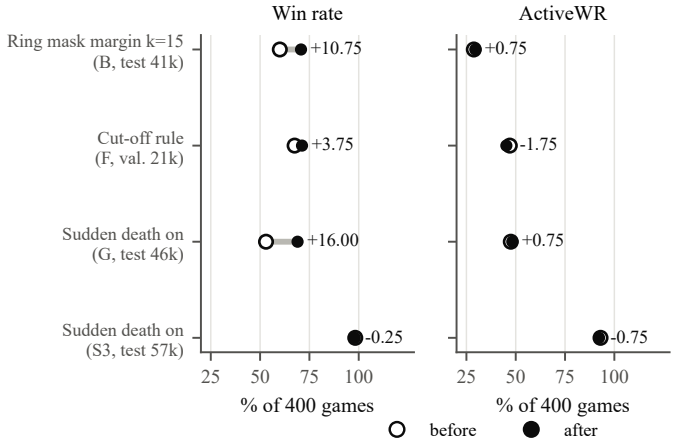


Fig. 8. Win rate versus ActiveWR for the same paired games. Labels give after-minus-before in percentage points. WR gains come from passive or ring wins.

relative advantages on decisive states (+0.75 [-5.75, 7.25] against the matched control), even though the latter raised the kill rate at those states by +7.12 [4.86, 9.72] pp; G is that checkpoint. Pushing BOMB at narrow states raised P(kill move) from 0.72 to 0.97 but not kills per game, and no KL weight separated "bomb at the right moment" from "bomb anywhere". Self-play RL against a pool of CNNs, starting from the distilled S2, erased the skill (ActiveWR 83.75 → 65.0 and 57.5; agreement with the search 97.4 → 75.6 and 73.1%), and a global KL anchor ($\beta = 1$) kept only 2.5 pp of it. Fig. 15 (Appendix) shows why RL from F is hard to read: rolling training ActiveWR wanders between 35 and 55% and validation peaks are not repeated on test.

E. RQ5: draws between strong agents

Against frozen CNNs (S2, G, S3) S3 still draws 53–64% of validation games, mostly at the 500-step limit (Fig. 11); against RuleBot it draws under 1% with a median game length of 105 steps (Fig. 14, Appendix). S3 improves on S2 against CNNs by only +3.33 [1.25, 5.50] ActiveWR (macro over three opponents) and −4.08 [-8.17, -0.17] draws. To ask whether these draws hide missed kills, we replayed 195 draws of S3 and ran the depth-3 search at every fifth state with two opponent models (Fig. 12). Under the RuleBot model the search sees a kill plan in 90.3% of draws (2.98% of states); with the opponent's own CNN as model it sees one in only 15.9% [9.9, 22.7] (0.19% of states), while all 20 control active wins contain a plan. RuleBot-style traps are therefore mostly illusory against a CNN. Starting the ring earlier (step 400, 350 or 300) did not help either: ActiveWR changed by −1.42 to −0.17 pp, and 80–87% of the remaining draws became simultaneous deaths in the last ring. Conversely, a search that always models the opponent by a CNN gains against CNNs (+12.0 against F) but loses 33 pp against RuleBot. Both agents use the same safety mask, so neither steps into a forecast blast; a CNN dodges in varied ways and almost always keeps an exit, so a forced kill rarely exists.

F. Transfer to an 11×11 board

Interpolating S3's fully connected layer to 11×11 gives 46.25% ActiveWR on the larger board, whereas S3 +

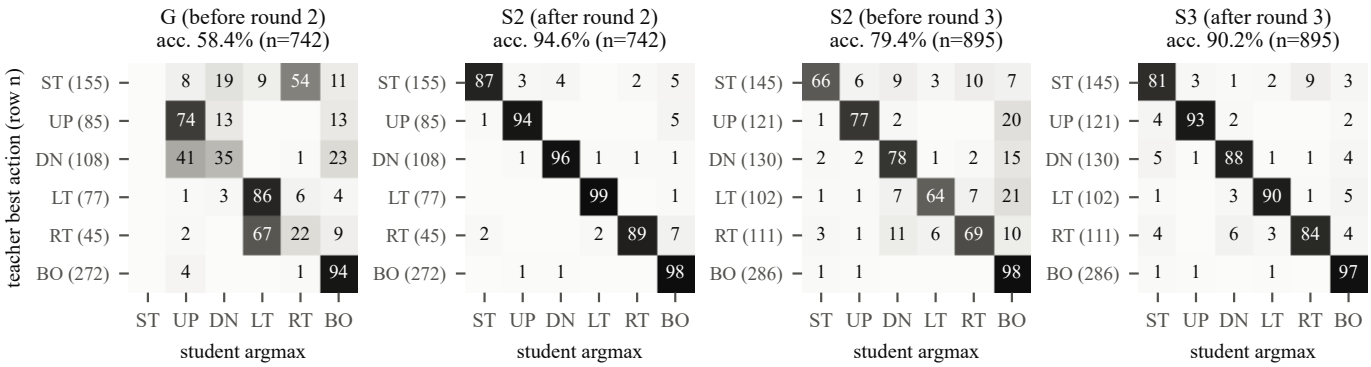


Fig. 9. Action confusion on held-out decision states with a unique best action (rows: teacher's best action, with the number of states; columns: student argmax; row-normalised %). Left pair: round 2 (teacher G+s2); right pair: round 3 (teacher S2+s3). ST/UP/DN/LT/RT/BO = STOP, UP, DOWN, LEFT, RIGHT, BOMB. Argmax on stored states, not sampled play; states with tied best actions (about 12%) are excluded.

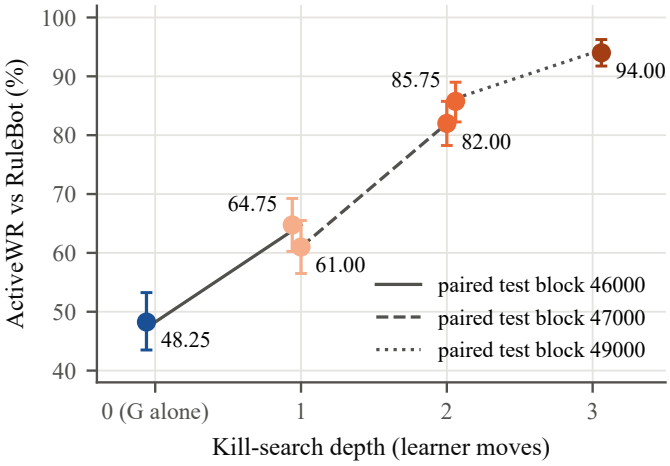


Fig. 10. ActiveWR versus kill-search depth around G. Marker colour identifies the agent; each line style joins the two agents tested on one paired block. Error bars: 95% CIs over boards.

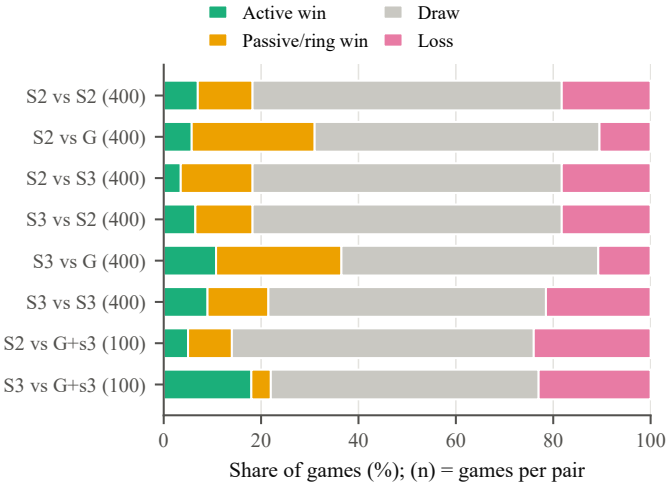


Fig. 11. Outcomes of CNN-versus-CNN and CNN-versus-search games on validation boards (learner listed first; games per pair in brackets). Outcome colours match Fig. 7; passive/ring wins are mostly the opponent caught by the ring.

depth-3 search reaches 98% on validation. One distillation round on 200 teacher games (746 decision and 1,310 anchor labels, 16 epochs) produced S3-11 with 87.25% on a fresh test block, +41.0 [34.5, 47.25], with self-kills +0.5 pp. Against a pool of frozen CNNs (S3, S2, G; 2,400 games) S3-11 scored 8.58 versus 4.75% macro ActiveWR, +3.83 [1.92, 5.83], below the gate, with draws at 64.33 versus 63.08%. Distillation transfers the RuleBot skill to a new board but does not resolve structural draws.

G. Free-for-all against several RuleBots

On a fresh block (seed 50000, schedule E, 500 steps) one learner played against RuleBots in free-for-all. With three agents on 9×9 (60 boards × 3 seats, 180 games), G reached 27.2 [21.1, 33.3] ActiveWR (40.6% WR) and G+s3 94.4 [90.6, 97.8] (98.9% WR). With four agents on 11 × 11 (40 boards × 4 seats, 160 games; the 9×9 network interpolated to the larger board), G reached 23.1 [16.9, 29.4] (36.3% WR, 53.8% draws) and G+s3 86.3 [80.0, 91.9] (88.8% WR, 10.6% draws). A RuleBot in the learner seat won 11.9% (6.3% ActiveWR) and killed itself in 35.6% of games.

The distilled students were then evaluated on a second fresh block (seed 59000, same protocol), with G re-run on the same boards so that the difference is paired. With three agents on 9×9, S3 reached 82.8 [76.1, 88.9] ActiveWR (87.8% WR, 7.2% draws, 0.6% self-kills) against 36.1 for G, a paired difference of +46.7 [+36.1, +56.7]. With four agents on 11 × 11, interpolating S3 gave 25.0 [18.1, 31.9], not clearly above G (17.5; +7.5 [-1.9, +16.9]), whereas the 11 × 11 student S3-11 (Section VI-F) reached 59.4 [50.6, 68.1] ActiveWR (63.8% WR, 25.0% draws), +41.9 [+31.9, +51.9] over G. Against fair shares of 33% (three agents) and 25% (four), 94% and 93% of the wins of S3 and S3-11 are active, against 64% and 58% for G: in free-for-all opponents also die by their own bombs, each other or the ring, so WR flatters weak agents most. Both students

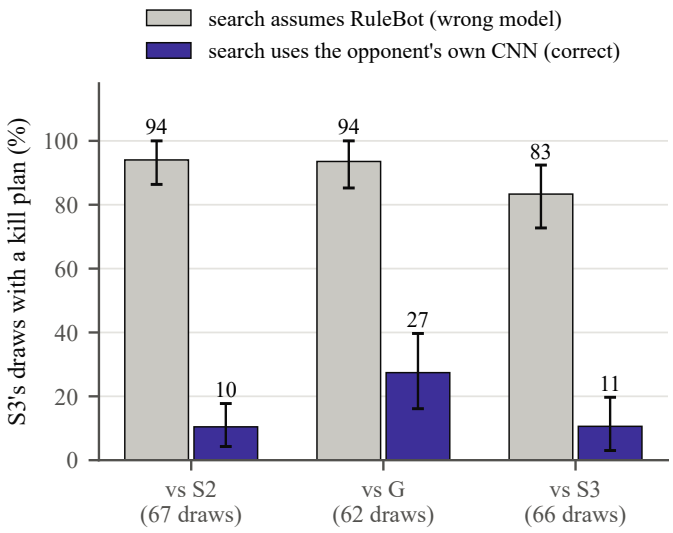


Fig. 12. Share of S3's draws against each CNN in which the depth-3 search finds a kill plan, under the wrong opponent model (RuleBot) and the correct one (the opponent's own CNN). Search runs at every fifth state. Error bars: 95% bootstrap CIs over games.

were distilled from 1v1 games only. G's ActiveWR differs between the two blocks (27.2 versus 36.1 on 9×9), so the students are compared with G+s3 only descriptively: the gap to G+s3 grows from 1.75 points in 1v1 to about 12 points in three-agent free-for-all (94.4 on block 50000).

VII. Discussion

Why dense distillation works where RL does not. Decisive states are rare (in a 230-step game, about 2.2 steps where the search scores differ and 1.25 with a kill), and Monte-Carlo policy gradient spreads credit over all steps [37]. A sparse label used once is too little signal; the same labels stored and replayed for several epochs, with counter-examples where BOMB is wrong, teach a conditional rule that RL's one-sided signal could not separate (Section VI-D). This matches the finding that more demonstrations matter more than better ones [23] and that policy distillation works best with sharp soft targets [26].

Why RL after distillation forgets. Our anchors and KL penalties used fixed weights and pointed to the old student. Kickstarting [29] needed a decaying weight to surpass the teacher, PPO found fixed-β KL hard to tune [40], and imitation-then-PPO in Pommerman forgot catastrophically [11]. Our measurement adds a reason: KL on decision states was about ten times larger than on ordinary states, so a global anchor is dominated by the states that do not matter. RL also mostly sharpened what F could already do: group-relative updates raised the kill rate by +7.12 pp at decisive states, and 86% of those states were ones F converted in some but not all of 8 replays. This is in line with the observation that RL raises majority-vote accuracy but not pass@k [41].

The ceiling. The search never uses the network, so expert iteration lacks ExIt's step where the apprentice improves the expert [19], [20]; S3 approaches but does not pass 94%. Promising next steps, not yet tested, are: aggregating data across rounds and sampling fewer correlated states per game [19], [28]; a search guided by S3's prior and value; pessimistic opponent scenarios [7] or a mixture of opponent models [42] instead of one RuleBot model, which our probe shows overfits (a joint-policy-correlation effect); and RL anchored to the search teacher on the student's own states with a decaying weight [29].

How far the results generalise. All agents were built and tested in one environment against one scripted opponent, and the search models that opponent. Three checks inside the environment show where the results hold. The student kept its skill in three-agent free-for-all, although it was distilled from one-versus-one games only (82.8% ActiveWR, Section VI-G). One more distillation round on 11×11 restored the skill on the larger board (+41.0 pp, Section VI-F). Against frozen CNN opponents, however, both the search and its students lose most of their advantage (Section VI-E). The study therefore supports a claim about turning a search that models a known opponent into a fast network. It does not support a claim about strength against arbitrary opponents. The most direct external test is the official Pommerman free-for-all against SimpleAgent, which needs kicking and a 10-step fuse in both the simulator and the search, plus new distillation rounds.

Draws. High draw rates between strong agents are common in Pommerman [7], [8], [15]. Our probe separates draws that hide a kill under the right opponent model from draws where none exists, and suggests that changing the rules (a final ring that does not fill the board, longer games) would matter more than a stronger agent.

Comparison with published agents. No agent was run in both environments, so Table VI positions results rather than ranking them. Pommerman is the broader benchmark (larger board, four agents, kicking, partial observation in team mode), but three differences make our numbers conservative. First, the criterion is stricter. Every published result counts all wins, including games in which opponents killed themselves; our headline number counts only kills credited to the agent's own bombs. Under the published criterion S3 would report 98.0% instead of 92.25%, and G+s3 88.8% instead of 86.3% in four-agent free-for-all. Second, waiting is not rewarded. Bombs explode after 8 steps instead of 10, and a closing ring shrinks the arena near the end, whereas in Pommerman most ties come from games that reach the 800-step limit [9]. Third, the budget is small. S3 was trained and distilled on consumer machines and needs one forward pass per move, 0.75 points below its search teacher; the strongest learning agent in Pommerman needed 200 million steps of population-based self-play and still wins only 30% against a depth-1 tree search [6], [7]. Within these caveats the ordering

TABLE VI
Reported results of published Bomberman/Pommerman agents and of ours. Settings differ (Pommerman: 11×11, bombs explode after 10 steps, kicking, 800-step games, partial observation in team mode; ours: no kicking, 8-step bombs, 500-step games with sudden death, full observation), so rows position results and are not a head-to-head comparison. No published work reports ActiveWR. FFA: free-for-all.

Work	Setting	Opponents	Method	Reported result
Resnick 2018 [3]	Pommerman FFA, 4 agents	3 SimpleAgents	reference level	an agent as strong as SimpleAgent wins ≈20%
Gao 2019 [4]	Pommerman FFA / team	3 SimpleAgents / SimpleAgent team	PPO + action pruning	WR 40–43% (FFA), 65–76% (team); 3×100 games
Perez-Liebana 2019 [15]	Pommerman FFA, full observation	3 rule-based agents	MCTS / RHEA	WR 46.5% / 33.0%
Weil 2023 [9]	official Pommerman FFA	3 SimpleAgents	MCTS + learned model (ExIt-style)	WR 61–78%, ties 12–34% (5 models, 100 games each)
Meisheri 2019 [11]	Pommerman team 2×2, partial	SimpleAgent team	imitation → PPO	WR 77.8%
Osogami 2019 [7]	Pommerman team 2×2	SimpleAgent team	depth-1 tree search, pessimistic scenarios	WR 92.6–99.7% (pessimism level 0–3)
Huynh 2024 [6]	Pommerman team 2×2, partial	SimpleAgent team; search agent dypm	PPO + LSTM, curriculum, population self-play, 200M steps	WR 98.85% vs SimpleAgent; 30.0% vs dypm
Ours, G	FFA 11×11, 4 agents	3 RuleBots	masked PG + group advantage	WR 36.3%, ActiveWR 23.1% (160 games)
Ours, G+s3	FFA 11×11, 4 agents	3 RuleBots	G + network-free kill-search	WR 88.8%, ActiveWR 86.3% (160 games)
Ours, S3-11	FFA 11×11, 4 agents	3 RuleBots	distilled CNN, no search at play time	WR 63.8%, ActiveWR 59.4% (160 games)
Ours, S3	FFA 9×9, 3 agents	2 RuleBots	distilled CNN, no search at play time	WR 87.8%, ActiveWR 82.8% (180 games)
Ours, G+s3	1v1 9×9	RuleBot	G + network-free kill-search	WR 99.5%, ActiveWR 94.0% (400 games)
Ours, S3	1v1 9×9	RuleBot	distilled CNN, no search at play time	WR 98.0%, ActiveWR 92.25% (400 games)

matches the Pommerman literature: search leads learning (our RL checkpoints reach 40.0–51.0% ActiveWR, G+s3 94.0%), and in the setting closest to Pommerman free-for-all (four agents on 11×11, Section VI-G) G+s3 wins 88.8% against three RuleBots and the network-only S3-11 63.8%, compared with 40–43% for PPO [4] and 61–78% for MCTS with learned models [9] against three SimpleAgents. Both reference bots are weak heuristics below the fair share (11.9% for RuleBot, about 20% at SimpleAgent level [3]) but were never played against each other, and our students were distilled and evaluated only against RuleBot. ActiveWR itself has no published counterpart; the closest is Skynet's observation that wins often came from opponents killing themselves [8].

VIII. Limitations

(1) Only distillation round 3 was replicated over training seeds (three seeds, all passing the gate); the earlier rounds and the RL arms used one seed, so their intervals measure variation over evaluation boards, not over training runs [34], [35]. (2) The reference opponent is RuleBot, and the search assumes RuleBot; S3 is strongest against it and only slightly better than S2 against CNNs. (3) Milestones in Table III come from different test blocks; one block varies by about ± 5 pp. B's row uses a cross-check block because its original test predates sudden death, and the 200-episode ablations from B were also evaluated without it. (4) Policies are sampled, so numbers reproduce bit-for-bit only with the same torch version; validation results from different torch versions are not compared. (5) Most results are 1v1 on 9×9; free-for-all and 11×11 results are single runs. (6) Some arms (F, S, S2, the self-play and KL-anchor runs) were trained on a second machine. (7) Of the 35 core references, one [14] was consulted only as an abstract, so its result is not listed in Table VI. (8) Our agents were not run in the official Pommerman environment (kicking, 800-step games, team mode with partial observation) or against SimpleAgent, so Table VI positions rather than compares; running S3 and G+s3 there is the most direct next test.

IX. Conclusion

In Bomberman, win rate rewards survival and opponent mistakes; ActiveWR measures what we want, an agent that wins by killing. Measured this way, model-free RL with a safety mask stalls below about 50%, while a short, network-free kill-search reaches 94%. Storing the search's labels on decision states and distilling them, three times over, gives a single CNN at 92.25% that needs one forward pass per move. Against equally careful neural opponents most games remain draws, and our probe indicates that this comes mainly from the game's rules.

Statements

Data and code availability. Environment, training, search and evaluation code, the pre-specified run registrations, per-game evaluation logs and the scripts that produce every figure (paper/scripts) are in the authors' repository. An anonymised copy is provided for review at https://anonymous.4open.science/r/ANONYMOUS-ID; the public repository will be linked in the camera-ready version.
Ethics. The study uses simulated games only and no human data.
Funding and competing interests. This work received no external funding; it was carried out as a student project. The authors declare no competing interests.
Author contributions (CRediT). Author 1 (first author): conceptualization, methodology, software, investigation, formal analysis, visualization, writing – original draft. Authors 2 and 3: [roles to be completed by the authors, e.g. software, investigation, validation, writing – review and editing]. All authors approved the final manuscript. Names are withheld for double-anonymous review.
Use of AI tools. An AI assistant was used to help draft and edit the text and to write plotting code; the authors designed the experiments, ran them, checked every number against the logs and take responsibility for the content.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529–533, 2015.

[2] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," Nature, vol. 575, pp. 350–354, 2019.

[3] C. Resnick, W. Eldridge, D. Ha, D. Britz, J. Foerster, J. Togelius, K. Cho, and J. Bruna, "Pommerman: A multi-agent playground," arXiv preprint arXiv:1809.07124, 2018.

[4] C. Gao, B. Kartal, P. Hernandez-Leal, and M. E. Taylor, "On hard exploration for reinforcement learning: A case study in Pommerman," in Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2019, arXiv:1907.11788.

[5] P. Peng, L. Pang, Y. Yuan, and C. Gao, "Continual match based training in Pommerman: Technical report," arXiv preprint arXiv:1812.07297, 2018.

[6] N.-M. Huynh, H.-G. Cao, and I.-C. Wu, "Multi-agent training for Pommerman: Curriculum learning and population-based self-play approach," arXiv preprint arXiv:2407.00662, 2024.

[7] T. Osogami and T. Takahashi, "Real-time tree search with pessimistic scenarios: Winning the NeurIPS 2018 Pommerman competition," in Proc. Asian Conf. on Machine Learning (ACML), ser. PMLR, vol. 101, 2019, arXiv:1902.10870.

[8] C. Gao, P. Hernandez-Leal, B. Kartal, and M. E. Taylor, "Skynet: A top deep RL agent in the inaugural Pommerman team competition," in Proc. Multidisciplinary Conf. on Reinforcement Learning and Decision Making (RLDM), 2019, arXiv:1905.01360.

[9] J. Weil, J. Czech, T. Meuser, and K. Kersting, "Know your enemy: Investigating Monte-Carlo tree search with opponent models in Pommerman," in Adaptive and Learning Agents Workshop at AAMAS (ALA), 2023, arXiv:2305.13206.

[10] B. Kartal, P. Hernandez-Leal, and M. E. Taylor, "Using Monte Carlo tree search as a demonstrator within asynchronous deep RL," arXiv preprint arXiv:1812.00045, 2018.

[11] H. Meisheri, O. Shelke, R. Verma, and H. Khadilkar, "Accelerating training in Pommerman with imitation and reinforcement learning," arXiv preprint arXiv:1911.04947, 2019.

[12] C. Resnick, R. Raileanu, S. Kapoor, A. Peysakhovich, K. Cho, and J. Bruna, "Backplay: "Man muss immer umkehren"," arXiv preprint arXiv:1807.06919, 2018.

[13] J. Groot Kormelink, M. M. Drugan, and M. A. Wiering, "Exploration methods for connectionist Q-learning in Bomberman," in Proc. Int. Conf. on Agents and Artificial Intelligence (ICAART), 2018, pp. 355–362.

[14] Í. Goulart, A. Paes, and E. Clua, "Learning how to play Bomberman with deep reinforcement and imitation learning," in Entertainment Computing and Serious Games (ICEC-JCSG 2019), ser. LNCS, vol. 11863. Springer, 2019, pp. 121–133, abstract only consulted.

[15] D. Perez-Liebana, R. D. Gaina, O. Drageset, E. İlhan, M. Balla, and S. M. Lucas, "Analysis of statistical forward planning methods in Pommerman," in Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), vol. 15, no. 1, 2019, pp. 66–72.

[16] F. Helfenstein, J. Czech, J. Blüml, M. Eisel, and K. Kersting, "Checkmating one, by using many: Combining mixture of experts with MCTS to improve in chess," arXiv preprint arXiv:2401.16852, 2024.

[17] J. Sun, S. Chen, C. Zhang, Y. Ma, and J. Zhang, "Decision-making with speculative opponent models," arXiv preprint arXiv:2211.11940, 2022.

[18] X. Guo, S. Singh, H. Lee, R. L. Lewis, and X. Wang, "Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning," in Advances in Neural Information Processing Systems (NIPS), vol. 27, 2014, pp. 3338–3346.

[19] T. Anthony, Z. Tian, and D. Barber, "Thinking fast and slow with deep learning and tree search," in Advances in Neural Information Processing Systems (NIPS), 2017, arXiv:1705.08439.

[20] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of Go without human knowledge," Nature, vol. 550, pp. 354–359, 2017.

[21] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "Mastering chess and shogi by self-play with a general reinforcement learning algorithm," arXiv preprint arXiv:1712.01815, 2017.

[22] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in Proc. Int. Conf. on Machine Learning (ICML), 2016, arXiv:1602.01783.

[23] B. Kartal, P. Hernandez-Leal, C. Gao, and M. E. Taylor, "Safer deep RL with shallow MCTS: A case study in Pommerman," arXiv preprint arXiv:1904.05759, 2019.

[24] B. Kartal, P. Hernandez-Leal, and M. E. Taylor, "Action guidance with MCTS for deep reinforcement learning," in Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2019, arXiv:1907.11703.

[25] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[26] A. A. Rusu, S. G. Colmenarejo, Ç. Gülçehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell, "Policy distillation," in Int. Conf. on Learning Representations (ICLR), 2016, arXiv:1511.06295.

[27] W. M. Czarnecki, R. Pascanu, S. Osindero, S. Jayakumar, G. Swirszcz, and M. Jaderberg, "Distilling policy distillation," in Proc. Int. Conf. on Artificial Intelligence and Statistics (AISTATS), ser. PMLR, vol. 89, 2019, pp. 1331–1340.

[28] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in Proc. Int. Conf. on Artificial Intelligence and Statistics (AISTATS), 2011, arXiv:1011.0686.

[29] S. Schmitt, J. J. Hudson, A. Zidek, S. Osindero, C. Doersch, W. M. Czarnecki, J. Z. Leibo, H. Kuttler, A. Zisserman, K. Simonyan, and S. M. A. Eslami, "Kickstarting deep reinforcement learning," arXiv preprint arXiv:1803.03835, 2018.

[30] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in Proc. Int. FLAIRS Conference, vol. 35, 2022, arXiv:2006.14171.
[31] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conf. on Artificial Intelligence, 2018, arXiv:1708.08611.
[32] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in Proc. Int. Conf. on Machine Learning (ICML), 1999, pp. 278–287.
[33] T. Bansal, J. Pachocki, S. Sidor, I. Sutskever, and I. Mordatch, "Emergent complexity via multi-agent competition," in Int. Conf. on Learning Representations (ICLR), 2018, arXiv:1710.03748.
[34] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proc. AAAI Conf. on Artificial Intelligence, 2018, arXiv:1709.06560.
[35] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems (NeurIPS), 2021, arXiv:2108.13264.
[36] B. Efron, "Bootstrap methods: Another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979.
[37] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol. 8, pp. 229–256, 1992.
[38] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.
[39] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in Int. Conf. on Learning Representations (ICLR), 2016, arXiv:1506.02438.
[40] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
[41] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv preprint arXiv:2402.03300, 2024.
[42] M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver, and T. Graepel, "A unified game-theoretic approach to multiagent reinforcement learning," in Advances in Neural Information Processing Systems (NIPS), 2017, arXiv:1711.00832.

Appendix

Figs. 13–15 show the observation planes, game lengths and RL training curves.

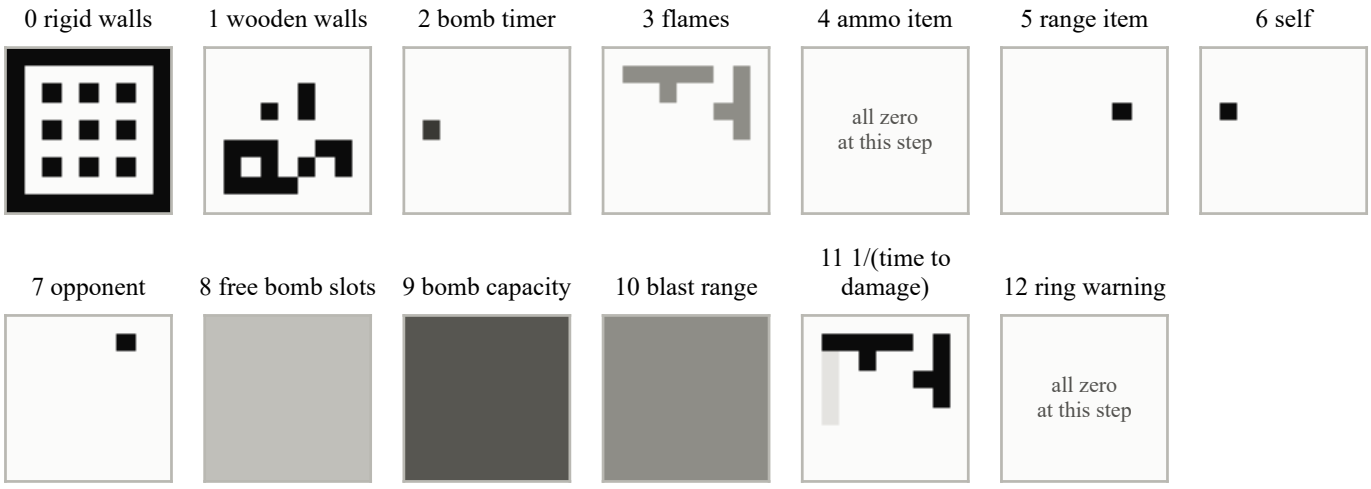


Fig. 13. The 13 observation planes for one agent (validation board 21007, step 26, RuleBot vs RuleBot; darker = larger). The agent has collected one ammo item (capacity 2 of 3) and has one bomb on the board, so one slot is free; a range item lies on the board. Planes marked "all zero" are empty at this step: no ammo item is visible, and the ring warning starts 40 steps before the first closure.

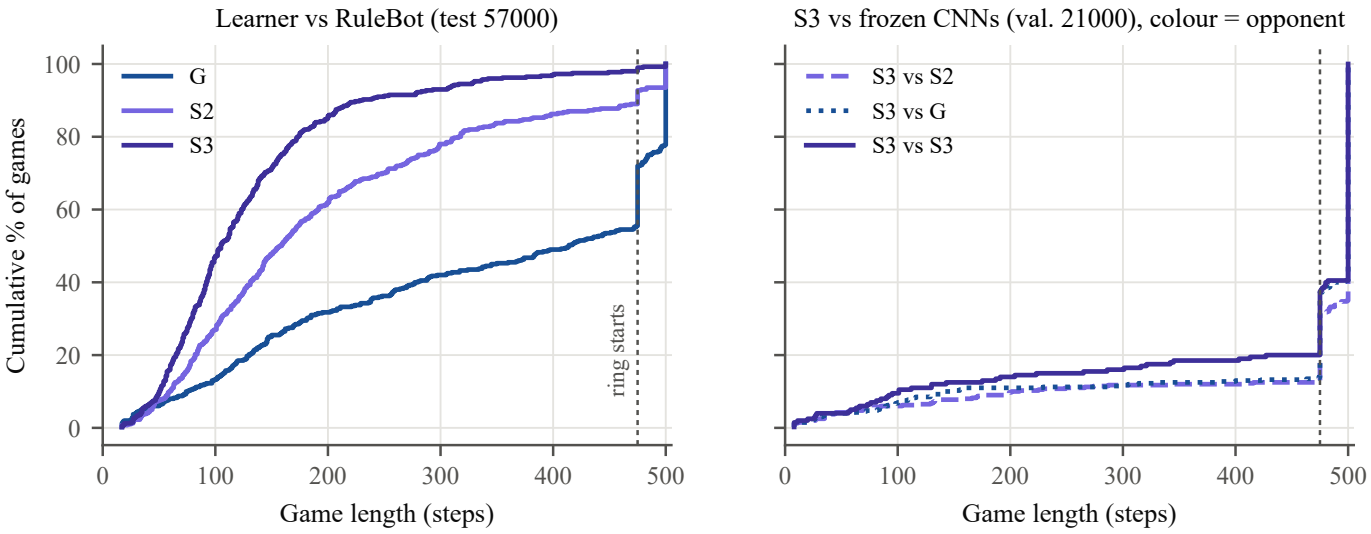


Fig. 14. Cumulative distribution of game length. Against RuleBot, stronger agents finish earlier (median 414, 158 and 105 steps for G, S2, S3; 22.25, 6.5 and 0.75% reach the limit). Against frozen CNNs (right; colour = opponent), 59.5–65% of S3's games reach 500 steps. The left panel uses test block 57000, the right one validation block 21000.

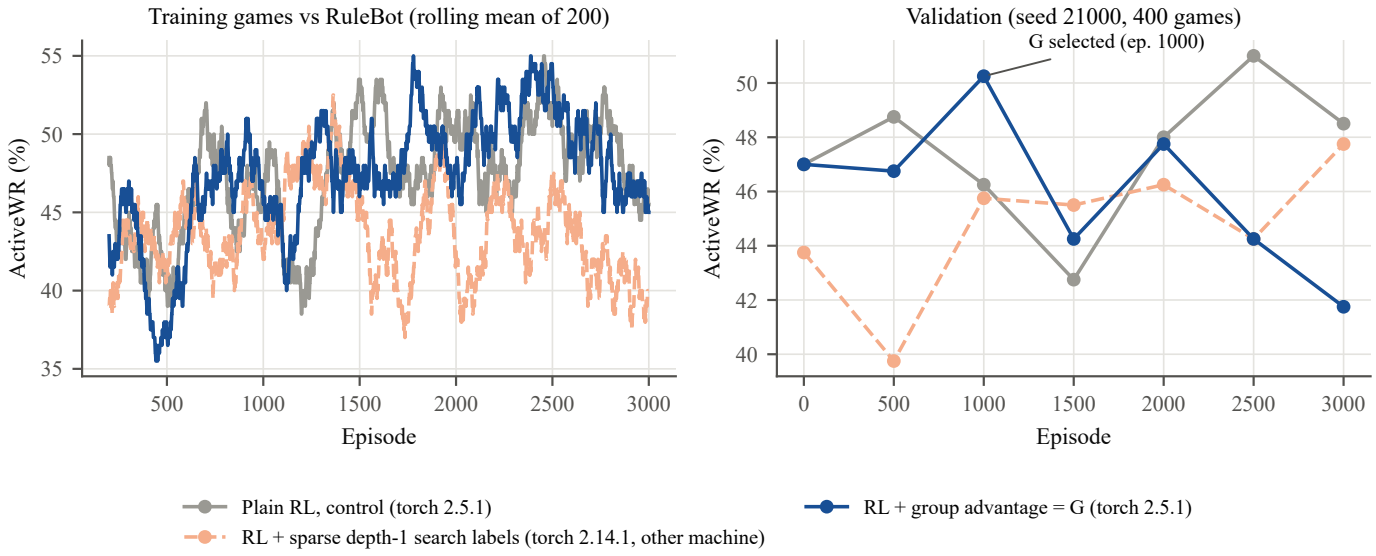


Fig. 15. RL from F (3,000 episodes per arm, one train seed). Left: rolling ActiveWR of training games against RuleBot. Right: validation ActiveWR every 500 episodes. The group-advantage arm peaked at episode 1000 (50.25) and became G; on the test block it was +0.75 over the control. The dashed arm ran on a second machine with torch 2.14.1; because actions are sampled, its validation numbers are not comparable with the other two (the same F scores 43.75 there versus 47.0 at episode 0). Its test result (Table IV) was re-run on the same machine as the others.